

Business Intelligence & Predictive Analytics for E-Commerce using Olist Data



QTREE TECHNOLOGIES

Student Registration Number	2026QT052
Name	Sridharan M
Course	Python with Data Analytics & ML Integration
Project Guide	Ms.Nithya Ravi
Institute	Qtree Technologies
Course Start Date	23.12.2025
Course End Date	02.05.2026
Project Submitted On	17.08.2026

Authorized signature

Trainer Signature

Table of Contents

Abstract.....	3
Introduction.....	4
Problem Statement	4
Objectives	4
Dataset Description.....	5
System Specification.....	6
Technology Stack.....	6
Data Flow Diagram.....	7
Data Modeling	8
Dashboard Development.....	8
Page 1 — Business Overview	8
Page 2 — Customer & Products	9
Page 3 — Logistics & Regions	10
Page 4 — AI & ML Insights	11
Page 5 — Strategic Insights	12
DAX Formulas.....	13
Python Integration.....	15
Machine Learning Model.....	15
Conclusion	26

Abstract

This project develops an end-to-end Business Intelligence and Predictive Analytics solution using the Brazilian E-Commerce Public Dataset by Olist. The solution integrates Python for data cleaning and customer RFM segmentation, MySQL for structured storage, Power BI for interactive reporting and DAX analysis, and embedded machine learning for short-term order and product demand prediction.

The analysis covers nearly 100,000 orders across nine related CSV datasets, enabling evaluation of revenue, customer behavior, product performance, payment methods, regional distribution, delivery efficiency, and review patterns. The Power BI report is organized into five analytical pages: Business Overview, Customer & Products, Logistics & Regions, AI & ML Insights, and Strategic Insights.

Machine learning is integrated directly into Power BI using scikit-learn LinearRegression models. One model forecasts monthly order demand using a previous-year order feature, while the second combines a linear trend with seasonal indices to estimate product-category demand for the next six months. Power BI Key Influencers and Decomposition Tree visuals further support AI-assisted revenue analysis.

The completed solution demonstrates how data engineering, business intelligence, customer analytics, artificial intelligence, and predictive modeling can be combined to transform fragmented e-commerce data into decision-ready insights. The project provides a practical foundation for customer retention, product planning, logistics improvement, and future predictive analytics.

Keywords: Olist E-Commerce, Data Analytics, Python, MySQL, Power BI, DAX, Machine Learning, Linear Regression, AI Visuals, Demand Forecasting.

Introduction

This document covers the Olist E-Commerce & Sales Analytics project — an end-to-end analytics solution on the public Olist Kaggle dataset. Data is cleaned in Python, staged in MySQL, modeled in Power BI, and delivered as a five-page dashboard with DAX KPIs, AI visuals, and ML forecasts.

Built end-to-end by a single analyst, it spans data engineering (Python, MySQL), modeling and DAX (Power BI), and applied ML (scikit-learn) — demonstrating job-ready Data Analyst skills in one connected workflow.

Problem Statement

The Olist dataset contains nearly 100,000 e-commerce orders spread across nine related CSV files. Without an integrated analytical view, it is difficult to evaluate sales performance, customer behavior, product demand, payment patterns, and delivery efficiency. This project addresses these gaps through Python, MySQL, Power BI, AI visuals, and machine learning.

Spread across nine raw CSV files, this data offers no consolidated view of performance — making revenue, delivery, loyalty, and demand questions hard to answer. This project turns it into a single, decision-ready analytics product.

Objectives

The objective is to transform nine Olist CSV datasets into a reliable analytical solution using Python, MySQL, Power BI, DAX, AI visuals, and machine learning. The project focuses on sales performance, customer segmentation, product and regional insights, logistics, and short-term demand prediction.

Key Challenges

- Integrating nine related CSV files with different data grains and keys.
- Handling missing values, inconsistent category labels, and date fields during preprocessing.
- Maintaining accurate Power BI relationships and DAX calculations across multiple fact-like tables.
- Producing meaningful ML forecasts from historical order patterns while keeping the visuals understandable.

Data Sources

The project uses the Brazilian E-Commerce Public Dataset by Olist from Kaggle, consisting of nine relational CSV files covering orders, order items, payments, customers, products, sellers, reviews, geolocation, and category translation data. Additional derived datasets include RFM-based customer segmentation and helper tables prepared for Power BI analysis.

Goal

The goal is to provide a single interactive analytics solution that helps stakeholders understand revenue, customers, products, regional performance, logistics, AI-driven revenue factors and future demand trends.

Dataset Description

The project uses the “Brazilian E-Commerce Public Dataset by Olist” from Kaggle — anonymized, relational CSV files linked through `order_id`, `customer_id/customer_unique_id`, `product_id`, and `seller_id`.

File / Table	Key Fields	Purpose in Project
orders	order_id, customer_id, order_status, order_purchase_timestamp, delivery dates	Central fact table; order lifecycle and delivery timing
order_items	order_id, product_id, seller_id, price, freight_value	Line-item revenue, product/seller linkage, freight cost
payments	order_id, payment_type, payment_value	Total revenue and payment-method mix
customers	customer_id, customer_unique_id, customer_city, customer_state	Customer identity, geography, segmentation join
products	product_id, product_category_name, dimensions/weight	Product catalog and category detail
sellers	seller_id, seller_city, seller_state	Seller location for logistics analysis
reviews	review_id, order_id, review_score	Customer satisfaction / rating and derived sentiment
geolocation	geolocation_zip_code_prefix, lat/lng	Geographic coordinates supporting the map visuals
category_translation	product_category_name, product_category_name_english	English category labels for reporting

Table 1: Source tables imported into the project

The model also includes derived tables: DataTable (calendar), RFM_Table and Customer_Segmentation (from the Python RFM analysis), and two helper tables — Customer_Order_Count and Order_Bucket_Sort — for visual ranking and sort order.

System Specification

Component	Specification
Device Name	LAPTOP-LH2L5JKG
Processor	Intel(R) Core(TM) i5-9300H CPU @ 2.40GHz (2.40 GHz)
Operating System Name	Microsoft Windows 11 Home Single Language
Installed RAM	8.00 GB (7.84 GB usable)
Graphics	NVIDIA GeForce GTX 1050 (4 GB) + Intel(R) UHD Graphics 630 (128 MB)
System Type	64-bit operating system, x64-based processor

Table 2: System specification

Technology Stack

Layer	Technology	Role in this Project
Data Source	Kaggle CSV files	Raw Olist e-commerce transactional data
Data Processing	Python 3.13.9, pandas, NumPy	Cleaning, merging, RFM feature engineering
Machine Learning	scikit-learn (LinearRegression, r2_score)	Order & product demand forecasting inside Power BI
Development Environment	Anaconda Navigator 2.7.0 (Jupyter Notebook)	Interactive scripting for ETL and RFM segmentation
Database	MySQL Workbench 8.0 CE	Central relational store for cleaned tables
DB Connectivity	mysql-connector-python, SQLAlchemy	Python-to-MySQL connection and bulk table loads (to_sql)
BI / Visualization	Power BI Desktop (2.152.1279.0, 64-bit)	Data modeling, DAX, dashboard authoring
Reporting / Review	Microsoft Excel	Quick data review, pivot checks, and CSV inspection

Table 3: Tools and technologies used across the project

Data Flow Diagram

The end-to-end data flow, from raw source files through processing, storage, and modeling to the final report, is summarized below.

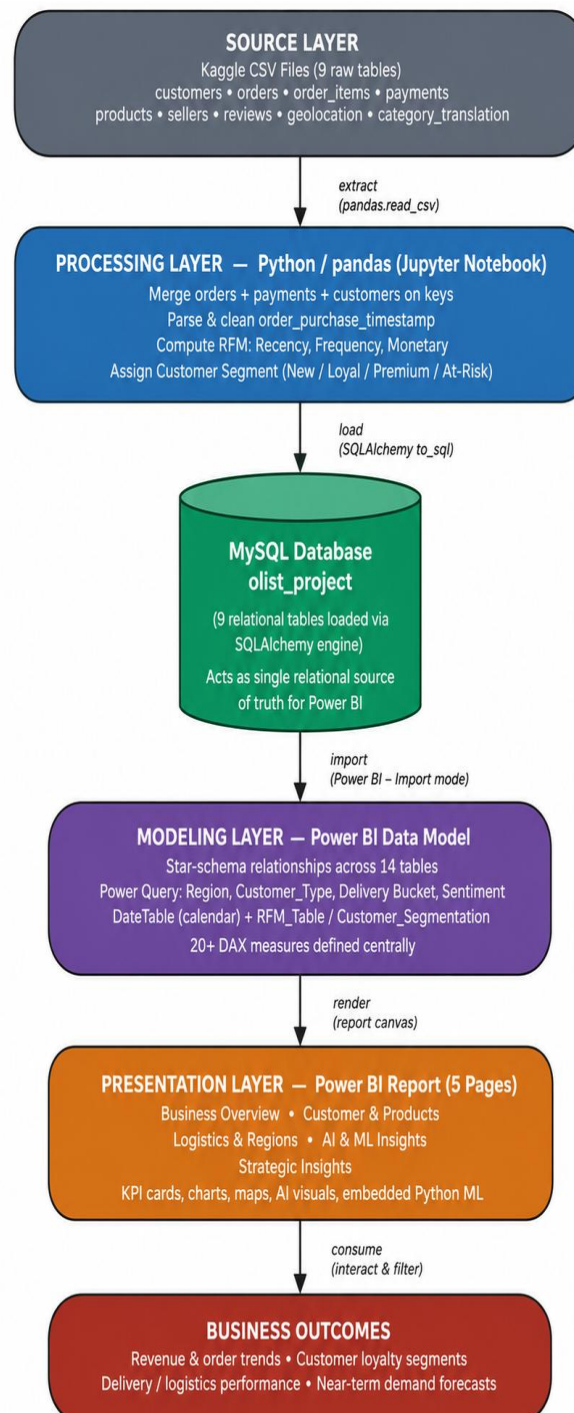


Figure 1: Data flow from raw source files to decision-ready dashboard

Data Modeling

Inside Power BI, the tables follow a star-schema layout: orders at the center, related to order_items, payments, and reviews via order_id, with customers, products, sellers, and category_translation as dimensions. DateTable drives all time-intelligence visuals.

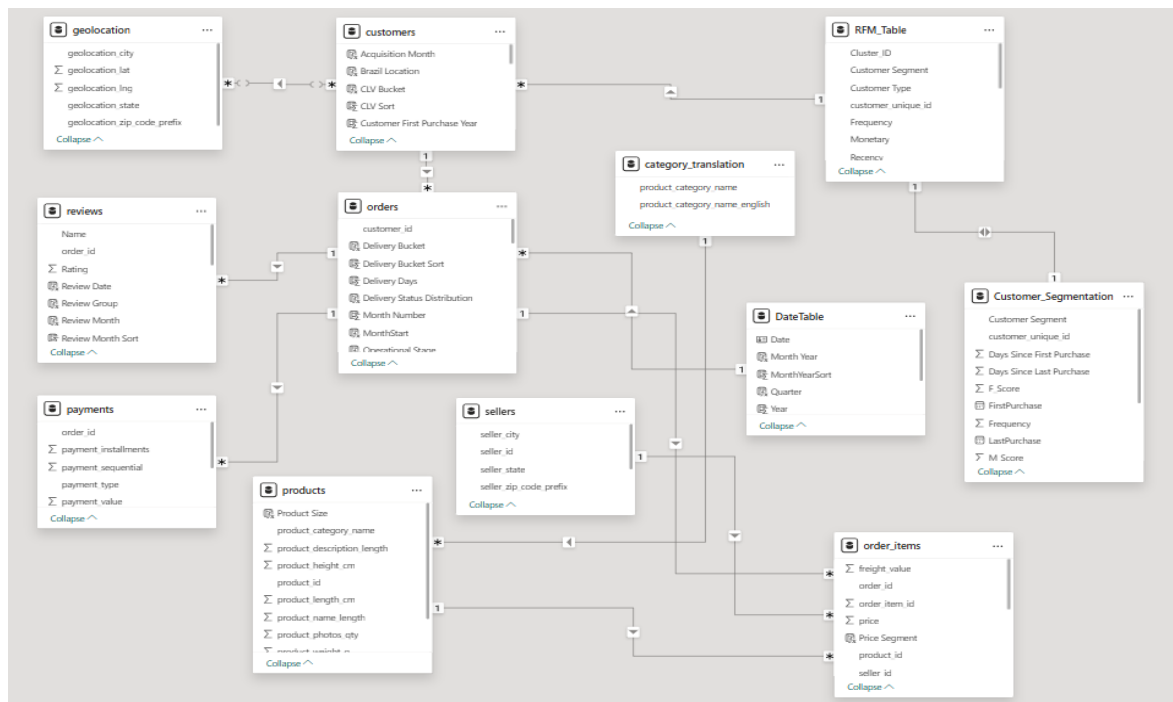


Figure 2: Data Schema

Dashboard Development

The report spans five pages, each with a consistent header, slicer panel, and KPI card strip, moving from a general overview to focused customer, logistics, AI, and strategic views.

Page 1 — Business Overview

KPI cards: Total Revenue, Total Orders, Unique Customers, Avg Order Value.

- Monthly Revenue Trend (line chart)
- Monthly Orders Trend (area chart)
- Top 10 Product Categories by Revenue (bar chart)
- Customer Growth Trend (column chart)
- Revenue by State (map)
- Revenue by Payment Type (donut chart)

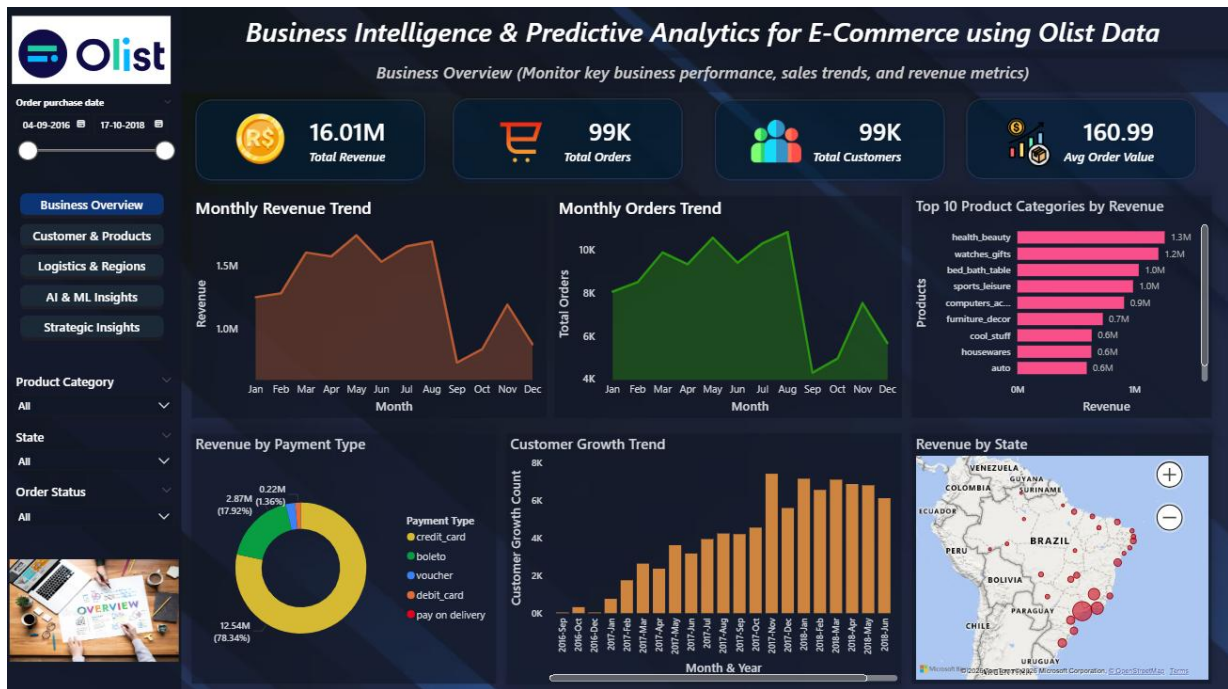


Figure 3: Business Overview Page

Page 2 — Customer & Products

KPI cards: Unique Customers, Repeat Customers, Avg Review Score, Total Products.

- Repeat vs One-Time Customers (donut chart)
- Customer Review Score Distribution (column chart)
- Avg Product Price by Category (bar chart)
- Top 8 Products by Units Sold (bar chart)
- Top 8 Products by Revenue (donut chart)
- Customer Distribution by Region (donut chart)

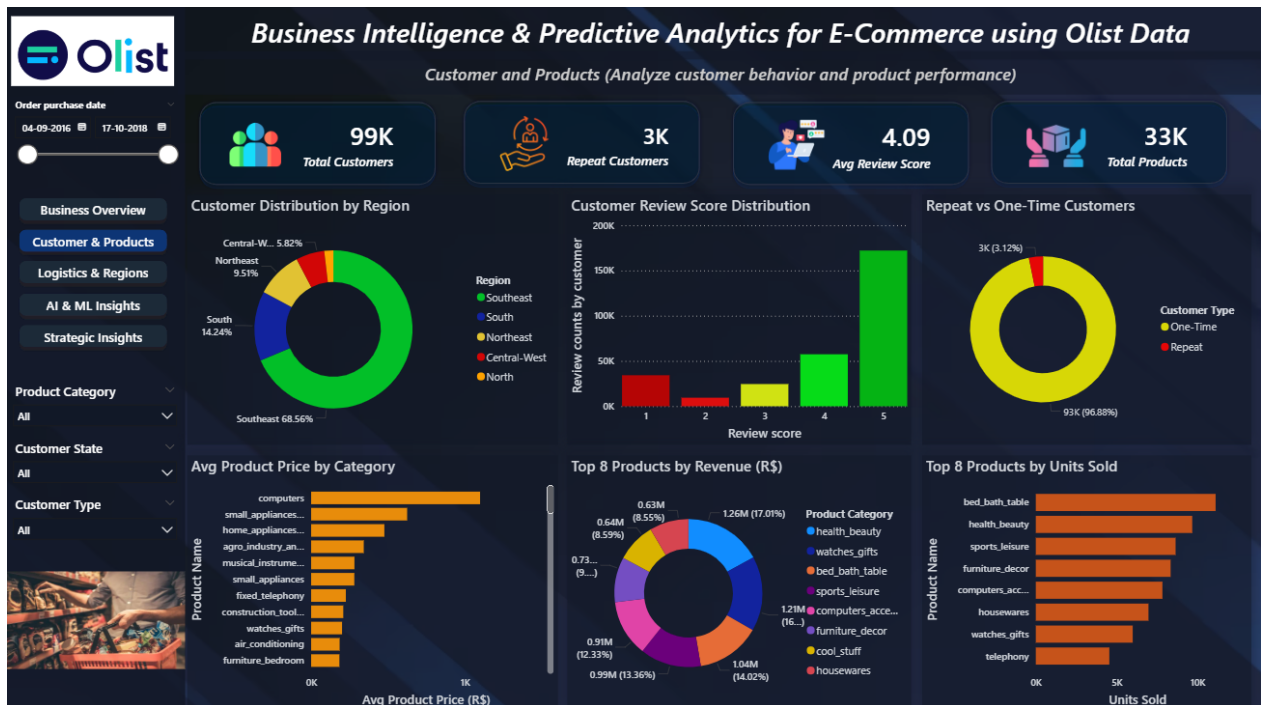


Figure 4: Customers & Products Page

Page 3 — Logistics & Regions

KPI cards: States Served, Avg Delivery Days, On-Time Delivery %, Avg Freight Cost.

- Delivery Performance by Region (pivot table)
- Delivery Performance by Location (map)
- Top 10 States by Revenue (bar chart)
- Delivery Time Analysis (column chart)
- Top 10 States with Highest Delivery Delays (bar chart)
- Estimated vs Actual Delivery Days (combo line/column chart)

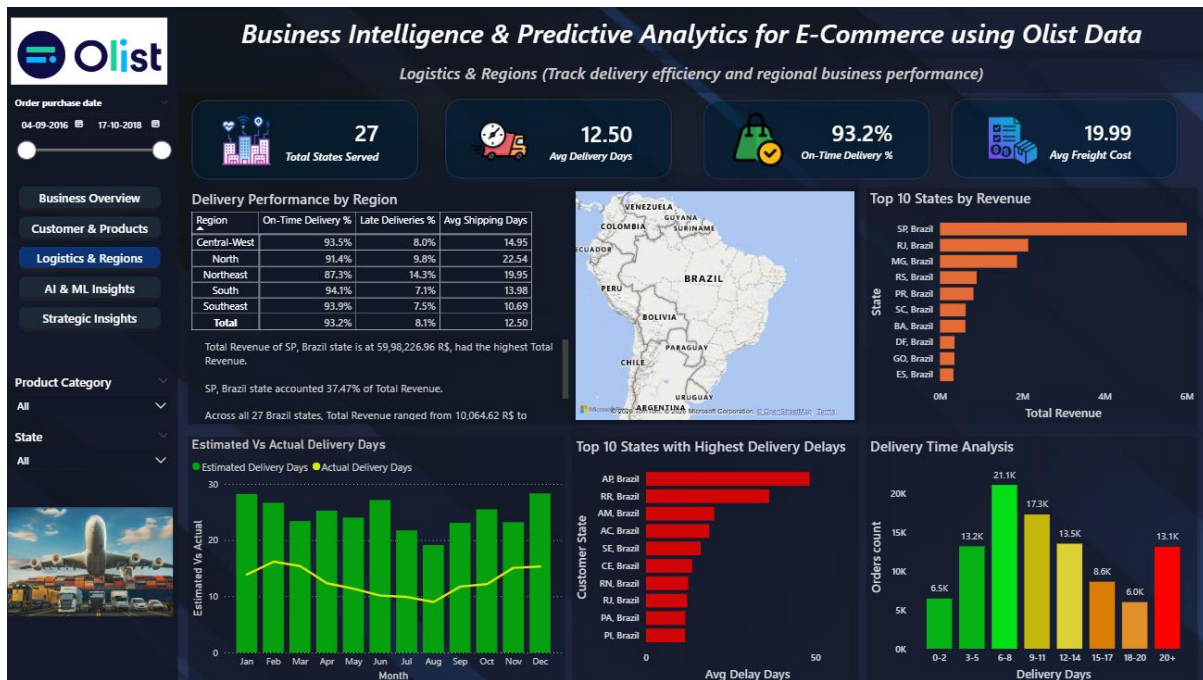


Figure 5: Logistics & Regions page

Page 4 — AI & ML Insights

KPI cards: Churn Risk Customers, Customer Lifetime Value, Premium Customers, High Risk Orders %.

- AI — Powered Revenue Insights (Key Influencers visual)
- AI — Revenue Breakdown Analysis (Decomposition Tree)
- ML — Order Demand Prediction (embedded Python visual, linear regression)
- ML — Product Demand Prediction by Category (embedded Python visual, trend + seasonality)

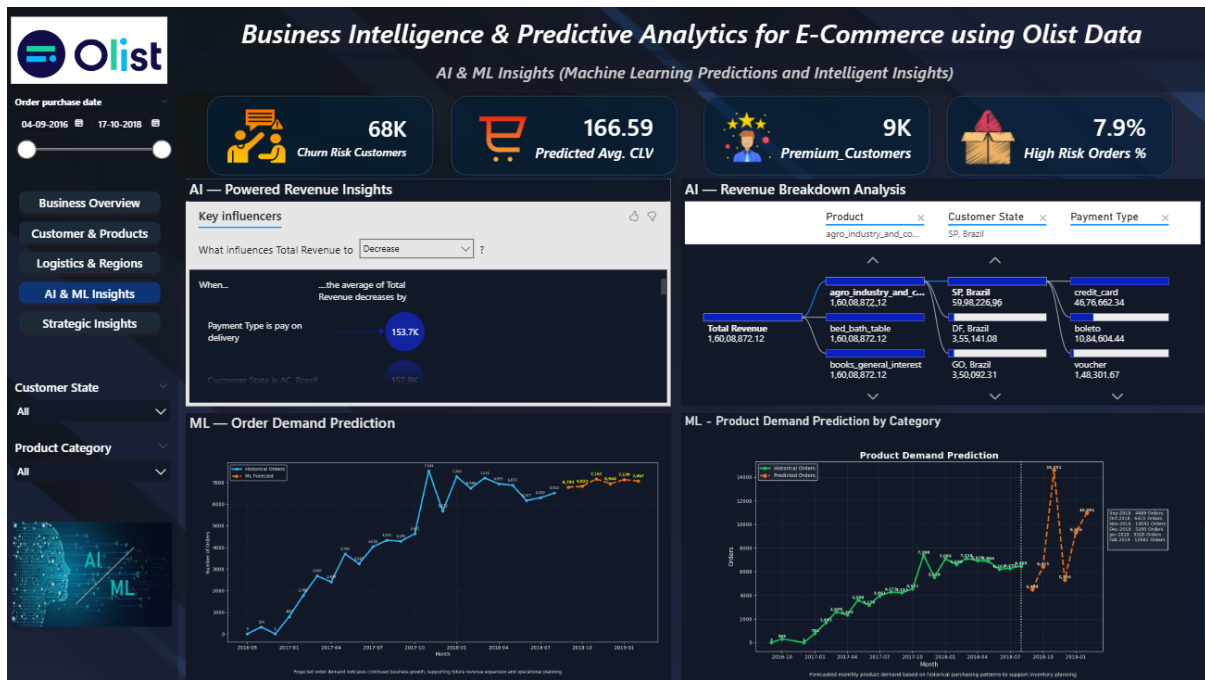


Figure 6: AI & ML Insights page

Page 5 — Strategic Insights

A summary page combining narrative textboxes with supporting imagery to translate prior findings into recommended actions (e.g. retention focus, regions needing logistics attention, demand trends).



Figure 7: Strategic Insights page

DAX Formulas

Revenue & Orders

Total Revenue =

SUM (payments[payment_value])

Total Orders =

DISTINCTCOUNT (orders[order_id])

Avg Order Value =

DIVIDE ([Total Revenue], [Total Orders])

Customers & Segmentation

Unique Customers =

DISTINCTCOUNT (customers[customer_unique_id])

Repeat Customers =

CALCULATE (
DISTINCTCOUNT (customers[customer_unique_id]),
FILTER (
VALUES (customers[customer_unique_id]),
CALCULATE (DISTINCTCOUNT (orders[order_id])) > 1
)
)

States Served =

DISTINCTCOUNT (customers[customer_state])

Avg Review Score =

AVERAGE (reviews[Rating])

Total Products =

DISTINCTCOUNT (products[product_id])

Churn Risk Customers =

CALCULATE (
DISTINCTCOUNT (RFM_Table[customer_unique_id]),
RFM_Table[Customer Segment] = "At Risk Customer"
)

Customer Lifetime Value =
AVERAGE (RFM_Table[Monetary])

Premium_Customers =
CALCULATE (
 DISTINCTCOUNT (Customer_Segmentation[customer_unique_id]),
 Customer_Segmentation[Customer Segment] = "Premium Customer"
)

Delivery & Logistics

Avg Delivery Days =
AVERAGEX (
 orders,
 DATEDIFF (orders[order_purchase_timestamp], orders[order_delivered_customer_date], DAY)
)

On-Time Delivery % =
VAR DeliveredOrders =
 CALCULATE ([Total Orders], NOT ISBLANK (orders[order_delivered_customer_date]))
VAR OnTimeOrders =
 CALCULATE (
 [Total Orders],
 orders[order_delivered_customer_date] <= orders[order_estimated_delivery_date]
)
RETURN
 DIVIDE (OnTimeOrders, DeliveredOrders)

Avg Freight Cost =
AVERAGE (order_items[freight_value])

High Risk Orders % =
DIVIDE (
 CALCULATE ([Total Orders], RFM_Table[Customer Segment] = "At Risk Customer"),
 [Total Orders]
)

Python Integration

Python plays two roles here: outside Power BI as the ETL and segmentation engine preparing data for MySQL, and inside Power BI as an embedded scripting engine rendering matplotlib charts driven by scikit-learn models.

Python for ETL and MySQL Connectivity

mysql-connector-python and SQLAlchemy connect the Jupyter Notebook to a local MySQL server and push cleaned DataFrames into olist_project:

MySQL Connectivity Code

```
import pandas as pd
import mysql.connector
from sqlalchemy import create_engine
conn = mysql.connector.connect(
    host="localhost", user="root", password="*****"
)
engine = create_engine(
    "mysql+mysqlconnector://root:*****@localhost/olist_project",
    pool_pre_ping=True
)
customers.to_sql(name='customers', con=engine,
    if_exists='replace', index=False)
```

Python Visuals inside Power BI

Power BI's native Python visual runs two scikit-learn regression models against the report's own data, rendering forecasts as matplotlib charts embedded in the canvas — keeping everything inside the .pbix file. Details in Machine Learning Model.

Machine Learning Model

Two lightweight forecasting models are embedded as Python visuals on the AI & ML Insights page, both using scikit-learn's LinearRegression for speed, transparency, and easy explanation.

Model 1 — ML: Order Demand Prediction

This script powers the “Order Demand Prediction” visual. It aggregates monthly order totals through August 2018, engineers a one-year lag feature (PrevYearOrders), and fits a LinearRegression model on it. Fit is checked with `r2_score`, then used to project six months (Sep 2018–Feb 2019) from six reference months. The result is a dark-themed matplotlib chart with historical and forecast lines plus an insight caption.

Order Demand Prediction Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
# =====
# DATA PREPARATION
# =====
dataset['order_purchase_timestamp'] = pd.to_datetime(
    dataset['order_purchase_timestamp'],
    errors='coerce'
)
dataset = dataset.dropna(
    subset=[
        'order_purchase_timestamp',
        'order_id'
    ]
)
# =====
# MONTHLY ORDER AGGREGATION
# =====
monthly_orders = dataset.groupby(
    dataset['order_purchase_timestamp'].dt.to_period('M')
)['order_id'].count().reset_index()
monthly_orders.columns = [
    'Month',
    'OrderCount'
]
monthly_orders['Month'] = monthly_orders['Month'].astype(str)
monthly_orders = monthly_orders[
    monthly_orders['OrderCount'] > 0
]
# =====
# KEEP DATA UP TO AUG-2018
# =====
```

```

monthly_orders = monthly_orders[
    monthly_orders['Month'] <= '2018-08'
].copy()
monthly_orders = monthly_orders.reset_index(
    drop=True
)
# =====
# CREATE MONTH NUMBER
# =====
monthly_orders['MonthNo'] = range(
    len(monthly_orders)
)
# =====
# CREATE LAG FEATURE
# SAME MONTH PREVIOUS YEAR
# =====
monthly_orders['PrevYearOrders'] = (
    monthly_orders['OrderCount']
    .shift(12)
)
ml_data = monthly_orders.dropna().copy()
# =====
# TRAINING DATA
# =====
X = ml_data[['PrevYearOrders']]
y = ml_data['OrderCount']
# =====
# MACHINE LEARNING MODEL
# =====
model = LinearRegression()
model.fit(X, y)
# =====
# MODEL EVALUATION
# =====
historical_predictions = model.predict(X)
r2 = r2_score(
    y,
    historical_predictions
)
# =====
# FORECAST MONTHS
# =====
reference_months = [
    '2017-09',
    '2017-10',

```

```

'2017-11',
'2017-12',
'2018-01',
'2018-02'
]
future_labels = [
'2018-09',
'2018-10',
'2018-11',
'2018-12',
'2019-01',
'2019-02'
]
forecast_orders = []
for month in reference_months:
    prev_orders = monthly_orders.loc[
        monthly_orders['Month'] == month,
        'OrderCount'
    ].values[0]
    prediction = model.predict(
        [[prev_orders]]
    )[0]
    forecast_orders.append(
        prediction
    )
forecast_orders = np.array(
    forecast_orders
)
# =====
# FUTURE INDEX
# =====
future_months = np.arange(
    len(monthly_orders),
    len(monthly_orders) + 6
)
# =====
# CHART
# =====
fig, ax = plt.subplots(
    figsize=(14, 7)
)
fig.patch.set_facecolor(
    '#111827'
)
ax.set_facecolor(

```

```

        '#111827'
    )
    # =====
    # HISTORICAL ORDERS
    # =====
    ax.plot(
        monthly_orders['MonthNo'],
        monthly_orders['OrderCount'],
        color='#38BDF8',
        linewidth=3,
        marker='o',
        markersize=5,
        label='Historical Orders'
    )
    # =====
    # FORECAST ORDERS
    # =====
    ax.plot(
        future_months,
        forecast_orders,
        color='#F97316',
        linestyle='--',
        linewidth=3,
        marker='o',
        markersize=6,
        label='ML Forecast'
    )
    # =====
    # HISTORICAL LABELS
    # =====
    for x, y_val in zip(
        monthly_orders['MonthNo'],
        monthly_orders['OrderCount']
    ):
        ax.annotate(
            f'{int(round(y_val)):.}',
            (x, y_val),
            textcoords='offset points',
            xytext=(0, 8),
            ha='center',
            color='white',
            fontsize=7
        )
    # =====
    # FORECAST LABELS

```

```

# =====
for x, y_val in zip(
    future_months,
    forecast_orders
):
    ax.annotate(
        f'{int(round(y_val)):.}',
        (x, y_val),
        textcoords='offset points',
        xytext=(0, 8),
        ha='center',
        color='yellow',
        fontsize=8,
        fontweight='bold'
    )
# =====
# AXIS LABELS
# =====
all_positions = (
    list(monthly_orders['MonthNo']) +
    list(future_months)
)
all_labels = (
    list(monthly_orders['Month']) +
    future_labels
)
step = max(
    1,
    len(all_labels) // 8
)
ax.set_xticks(
    all_positions[::step]
)
ax.set_xticklabels(
    all_labels[::step],
    color='white'
)
# =====
# TITLES
# =====
ax.set_xlabel(
    'Month',
    color='white'
)
ax.set_ylabel(

```

```

        'Number of Orders',
        color='white'
    )
    # =====
    # GRID
    # =====
    ax.grid(
        linestyle='--',
        alpha=0.15
    )
    # =====
    # TICKS
    # =====
    ax.tick_params(
        axis='x',
        colors='white'
    )
    ax.tick_params(
        axis='y',
        colors='white'
    )
    # =====
    # BORDER
    # =====
    for spine in ax.spines.values():
        spine.set_color('white')
    # =====
    # LEGEND
    # =====
    legend = ax.legend(
        facecolor='#111827',
        edgecolor='white'
    )
    for text in legend.get_texts():
        text.set_color('white')
    # =====
    # INSIGHT
    # =====
    fig.text(
        0.5,
        0.02,
        'Projected order demand indicates continued business growth, supporting future revenue expansion and operational
        planning.',
        ha='center',
        color='white',

```

```

    fontsize=9
)
# =====
# LAYOUT
# =====
plt.subplots_adjust(
    left=0.08,
    right=0.98,
    bottom=0.15,
    top=0.90
)
plt.show()

```

Result

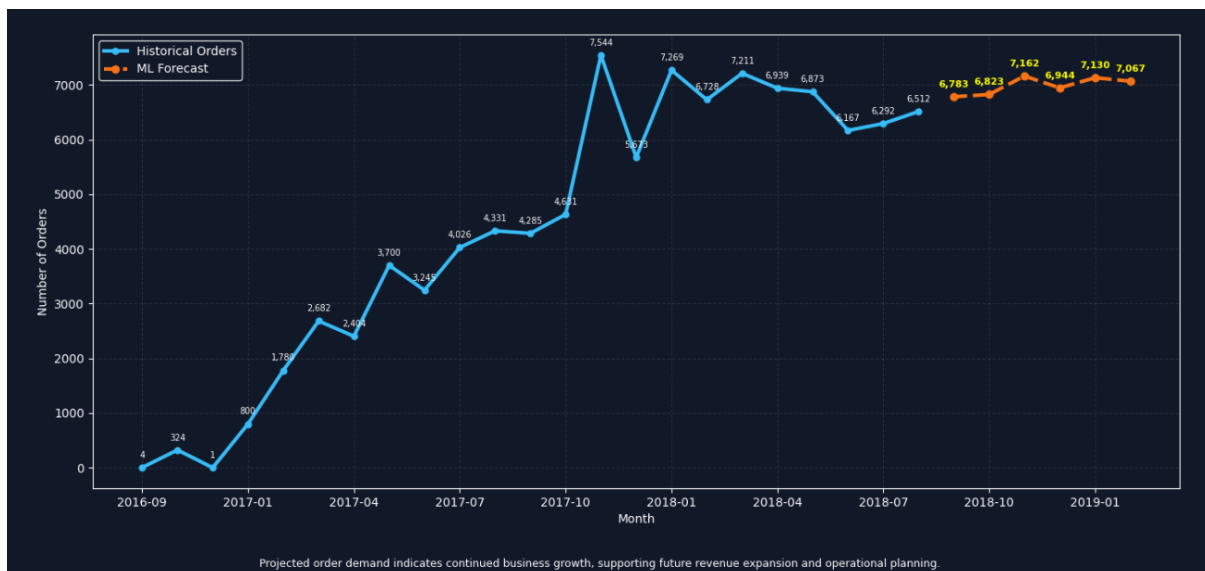


Figure 8: ML - Order Demand Prediction chart

Model 2 — ML: Product Demand Prediction by Category

This second script drives the “Product Demand Prediction by Category” visual for the selected category. Monthly counts are indexed and fit with a LinearRegression trend line, then a seasonal index (actual ÷ trend, averaged by month) captures recurring patterns like a November peak. The next six months are forecast by combining trend and seasonality, rendered as a dark-themed matplotlib chart with historical/forecast series and captions.

Product Demand Prediction by Category Code

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# =====

```

```

# LOAD DATA FROM POWER BI
# =====
df = dataset.copy()
df = df.dropna()
df["order_purchase_timestamp"] = pd.to_datetime(
    df["order_purchase_timestamp"],
    errors="coerce"
)
df = df.dropna(subset=["order_purchase_timestamp"])
# =====
# MONTHLY ORDER COUNT
# =====
df["Month"] = (
    df["order_purchase_timestamp"]
    .dt.to_period("M")
    .dt.to_timestamp()
)
monthly = (
    df.groupby("Month")
    .agg(Orders=("order_id", "count"))
    .reset_index()
)
monthly = monthly[monthly["Month"] <= pd.Timestamp("2018-08-01")]
monthly = monthly.sort_values("Month").reset_index(drop=True)
monthly["MonthIndex"] = np.arange(1, len(monthly) + 1)
monthly["CalMonth"] = monthly["Month"].dt.month # 1-12, used for seasonality
X = monthly[["MonthIndex"]]
y = monthly["Orders"]
# =====
# TREND MODEL (same as before)
# =====
trend_model = LinearRegression()
trend_model.fit(X, y)
monthly["Trend"] = trend_model.predict(X)
# =====
# SEASONAL INDEX (NEW)
# Ratio of Actual / Trend, averaged per calendar month
# This captures "same month last 1-2 years" behaviour
# =====
monthly["Seasonal Ratio"] = monthly["Orders"] / monthly["Trend"]
seasonal_index = (
    monthly.groupby("CalMonth")["Seasonal Ratio"]
    .mean()
)
# Normalize so seasonal indices average to 1 (keeps trend level intact)

```

```

seasonal_index = seasonal_index / seasonal_index.mean()
# =====
# FUTURE MONTHS
# =====
future_months = 6
future_index = np.arange(
    len(monthly) + 1,
    len(monthly) + future_months + 1
).reshape(-1, 1)
last_month = monthly["Month"].max()
future_dates = pd.date_range(
    last_month + pd.offsets.MonthBegin(1),
    periods=future_months,
    freq="MS"
)
# Base trend prediction for future months
future_trend = trend_model.predict(future_index)
# Apply seasonal index matching each future month's calendar month
future_cal_months = future_dates.month
seasonal_factors = np.array([
    seasonal_index.get(m, 1.0) for m in future_cal_months
])
future_pred = future_trend * seasonal_factors
future_pred = np.maximum(future_pred, 0)
forecast = pd.DataFrame({
    "Month": future_dates,
    "Predicted Orders": future_pred
})
# =====
# VISUALIZATION
# =====
fig, ax = plt.subplots(figsize=(14, 7))
fig.patch.set_facecolor("#111827")
ax.set_facecolor("#111827")
ax.plot(
    monthly["Month"], monthly["Orders"],
    color="#22C55E", linewidth=3, marker="o",
    label="Historical Orders"
)
ax.plot(
    forecast["Month"], forecast["Predicted Orders"],
    color="#F97316", linewidth=3, linestyle="--", marker="D",
    label="Predicted Orders"
)
for x, yv in zip(monthly["Month"], monthly["Orders"]):

```

```

    ax.text(x, yv + 120, f'{int(yv):,}', color="#BBF7D0",
            fontsize=8, ha="center", fontweight="bold")
for x, yv in zip(forecast["Month"], forecast["Predicted Orders"]):
    ax.text(x, yv + 120, f'{int(yv):,}', color="#FED7AA",
            fontsize=8, ha="center", fontweight="bold")
ax.axvline(x=last_month, color="white", linestyle=":", linewidth=2)
category = df["product_category_name_english"].iloc[0]
ax.set_title("Product Demand Prediction", fontsize=18, color="white", weight="bold")
ax.set_xlabel("Month", color="white", fontsize=12)
ax.set_ylabel("Orders", color="white", fontsize=12)
ax.grid(linestyle="--", alpha=0.25)
ax.tick_params(colors="white")
for spine in ax.spines.values():
    spine.set_color("white")
legend = ax.legend(facecolor="#1F2937", edgecolor="white", fontsize=10)
for txt in legend.get_texts():
    txt.set_color("white")
text = ""
for i in range(len(forecast)):
    text += (
        forecast.iloc[i]["Month"].strftime("%b-%Y")
        + " : "
        + str(int(round(forecast.iloc[i]["Predicted Orders"])))
        + " Orders\n"
    )
ax.text(
    1.02, 0.55, text,
    transform=ax.transAxes, fontsize=9, color="white",
    bbox=dict(facecolor="#1F2937", edgecolor="white")
)
fig.text(
    0.5, 0.02,
    "Forecasted monthly product demand based on historical purchasing patterns to support inventory planning",
    ha="center", color="white", fontsize=10
)
plt.subplots_adjust(right=0.82, bottom=0.12)
plt.show()

```

Result

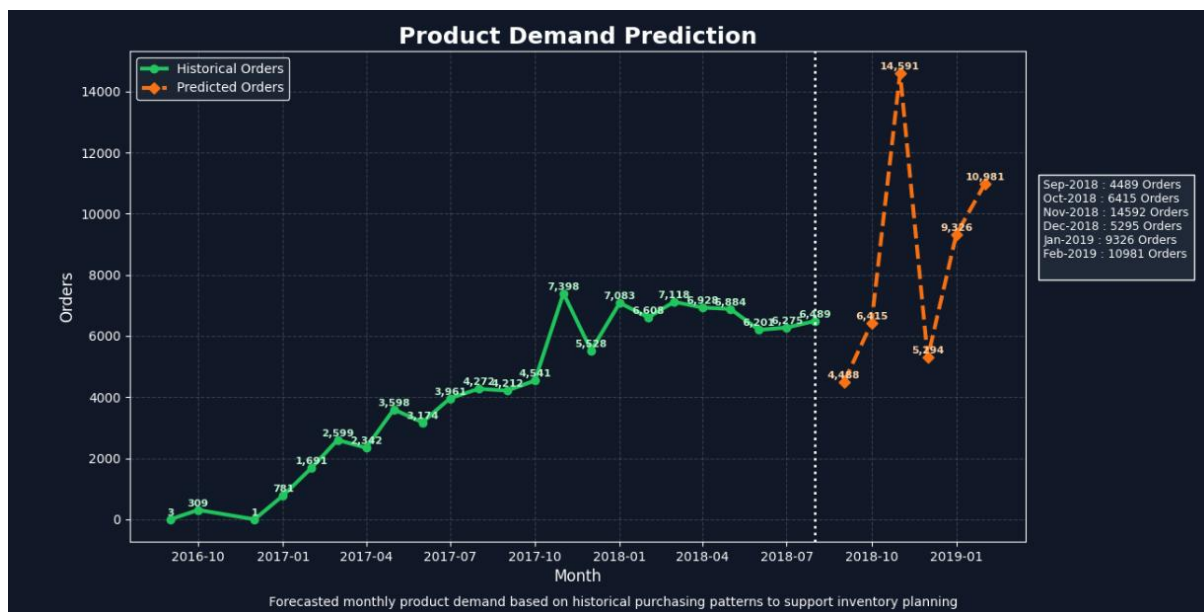


Figure 9: ML – Product Demand Prediction chart

AI Visuals (No-Code Modeling)

The page also uses Power BI's built-in Key Influencers (“AI — Powered Revenue Insights”) to surface top revenue drivers, and a Decomposition Tree (“AI — Revenue Breakdown Analysis”) for drill-down — no code or DAX required.

Conclusion

Project Summary

This project presents an end-to-end business intelligence and predictive analytics solution using the Brazilian E-Commerce Public Dataset by Olist. The workflow combines Python-based data preparation and RFM customer segmentation, MySQL data storage, Power BI data modeling and DAX, native AI visuals, and embedded machine learning forecasts to transform raw e-commerce transactions into decision-ready insights.

Key Findings

The dashboard records approximately ₹16.01M in total revenue, 99K orders, 99K customers, and an average order value of 160.99. Credit-card payments contribute the largest revenue share, health_beauty is the leading revenue category, and the AI & ML page highlights a 68K churn-risk customer segment, 166.59 predicted average CLV, 9K premium customers, and 7.9% high-risk orders.

Recommendations

Prioritize retention campaigns for at-risk customers, strengthen repeat-purchase programs, monitor high-revenue product categories, and use regional delivery-delay insights to improve logistics planning. Payment and product-category trends should also be used to guide promotional and inventory decisions.

Future Scope

Extend the solution with automated data refresh, real-time or near-real-time dashboards, richer customer propensity models, longer-horizon forecasting, additional ML algorithms, and cloud-based deployment for scalable analytics.

Final Conclusion

Overall, the project successfully demonstrates an integrated analytics pipeline that combines historical business reporting with customer segmentation, AI-assisted analysis, and machine learning-based demand prediction. The resulting Power BI solution provides a clear foundation for data-driven e-commerce decision-making and future predictive enhancements.